

CSE 20 Winter 2026

Exam Practice

- 1. Modeling, Sets and Functions, Algorithms** In machine learning, clustering can be used to group similar data for prediction and recommendation. For example, each Netflix user's viewing history can be represented as a n -tuple indicating their preferences about movies in the database, where n is the number of movies in the database. Each element in the n -tuple indicate the user's rating of the corresponding movie: 1 indicates the person liked the movie, -1 that they didn't, and 0 that they didn't rate it one way or another.

Consider the following algorithm for determining if a user's viewing history represents strong opinions on many movies.

Determine if user's ratings tuple encode strong opinions

```
1 procedure opinion(( $r_1, \dots, r_n$ ): a  $n$ -tuple of ratings;  $c$ : a nonnegative integer)
2    $sum := 0$ 
3   for  $i := 1$  to  $n$ 
4     if  $r_i \neq 0$ 
5        $sum := sum + 1$ 
6   return  $sum \geq c$ 
```

- When $n = 3$, describe the set of all n -tuples representing user ratings
 - By the roster method.
 - With set builder notation.
 - Using a recursive definition.
- What are the possible return (output) values of this algorithm?
- Trace** the computation of $opinion((-1, 0, 0, 0, 1, 1, -1), 2)$.
- Give an example of a nonnegative integer c so that, no matter which n -tuple $(r_1, \dots, r_n)$ we consider, $opinion((r_1, \dots, r_n), c)$ will have the same value. What value is it?

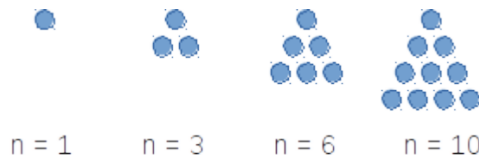
2. Sets and Functions

- Give a recursive definition for the set $\mathbb{N}$.
- Use the recursive definition from part (a) to give a recursive definition for the function with domain $\mathbb{N}$, codomain $\mathbb{N}$ which computes, for input i , the sum of the first i powers of 2. For example, on input 0, the function evaluates to 2^0 , namely to 1. On input 1, the function evaluates to $2^0 + 2^1$, namely 3. On input 2, the function evaluates to $2^0 + 2^1 + 2^2$, namely 7.

3. Base expansions, Multiple representations

1. Compute the ternary (base 3) expansion of 28.
2. Compute the product of $(6A)_{16}$ and $(11)_{16}$, without converting either number to another base.
3. Confirm your answer for part (b) by converting $(6A)_{16}$ and $(11)_{16}$ to decimal, multiplying them, and converting the product to base 16.
4. How many bits will there be in the binary (base 2) expansion of 2020? Can you compute this without fully converting 2020 to base 2?
5. Give an example of a number that can be represented in base 2 fixed-width 3, but not in base 2 expansion.
6. Give an example of a number that can be represented in base 2 expansion, but not in base 2, fixed-width 3 expansion.
7. Give the representation of -7 in sign-magnitude width 3 and 2s complement width 3. Then do the same for width 4.
8. Give the representation of 10.5625 in fixed-width base-2 expansion with integer width 4 and fractional width 9.

- 4. Circuits** A triangular number (or triangle number) counts the objects that can form an equilateral triangle, as in the diagram below. The n^{th} triangular number is the sum of the first n integers, as shown in the following figure illustrating the first four triangular numbers (what is the fifth one?):



Design a circuit that takes four inputs x_0, x_1, x_2, x_3 and outputs True (T or 1) if the integer value $(x_3x_2x_1x_0)_{2,4}$ is a triangular number, and False (F or 0) otherwise. You may assume that 0 is not a triangular number. (*Credit: UBC Department of Computer Science*)

5. Circuits, Compound Propositions, Logical Equivalence

1. Draw a logic circuit that uses **exactly three** gates and is logically equivalent to

$$q \leftrightarrow (p \wedge r)$$

You may (only) use AND, OR, NOT, and XOR gates.

2. Write a compound proposition which is logically equivalent to

$$(p \oplus q) \leftrightarrow r$$

You may only use the logical operators negation ($\neg$), conjunction ($\wedge$), and disjunction ($\vee$).

3. Find a compound proposition that is in DNF (disjunctive normal form) and is logically equivalent to

$$(p \vee q \vee \neg r) \wedge (p \vee \neg q \vee r) \wedge (\neg p \vee q \vee r)$$

6. Translating Propositional Logic

p is “The display is 13.3-inch”

r is “There is at least 128GB of flash storage”

u is “There is at least 512GB of flash storage”

q is “The processor is 2.2 GHz”

s is “There is at least 256GB of flash storage”

1. Are the statements

$$p \rightarrow (r \vee s \vee u) \quad , \quad q \rightarrow (s \vee u) \quad , \quad p \leftrightarrow q \quad , \quad \neg u$$

consistent? If so, translate to English a possible assignment of truth values to the input propositions that makes all four statements true simultaneously.

2. Consider this statement in English:

It’s not the case that both the display is 13.3-inch and the processor is 2.2 GHz.

Determine whether each of the compound propositions below is equivalent to the **negation** of that statement, and justify your answers using either truth tables or other equivalences.

Possible compound propositions:

- $\neg p \vee \neg q$
- $\neg(p \rightarrow \neg q)$
- $\neg(p \wedge q)$
- $(\neg p \leftrightarrow \neg q) \wedge p$

3. Consider the compound proposition

$$(p \wedge q) \rightarrow (r \vee s \vee u)$$

Express the **contrapositive** of this conditional as a compound proposition.

Then, give an assignment of truth values to each of the input propositional variables for which the original compound proposition is True but its **converse** is False.

7. Evaluating predicates, Evaluating nested predicates

1. Over the domain $\{1, 2, 3, 4, 5\}$ give an example of predicates $P(x), Q(x)$ which demonstrate that

$$(\forall x P(x)) \vee (\forall x Q(x)) \quad \text{is not logically equivalent to} \quad \forall x (P(x) \vee Q(x))$$

2. Over the domain $\{0, 1, 2\}$, give an example of predicates $P(x), Q(x)$ for which all of these statements are true:

$$\forall x (P(x) \rightarrow Q(x))$$

$$\exists x P(x)$$

$$\exists x \neg P(x)$$

$$\exists x \neg Q(x)$$

- 8. Evaluating predicates, Evaluating nested predicates** Recall that S is defined as the set of all RNA strands, where each strand is a nonempty string made of the bases in $B = \{\mathbf{A}, \mathbf{U}, \mathbf{G}, \mathbf{C}\}$. Recall the definition of the following predicates: $L_{\mathbf{A}}$ with domain S is defined recursively by:

$$\text{Basis step: } L_{\mathbf{A}}(\mathbf{A}) = T, L_{\mathbf{A}}(\mathbf{C}) = L_{\mathbf{A}}(\mathbf{G}) = L_{\mathbf{A}}(\mathbf{U}) = F$$

$$\text{Recursive step: If } s \in S \text{ and } b \in B, \text{ then } L_{\mathbf{A}}(sb) = L_{\mathbf{A}}(s)$$

P_{AUC} with domain S is defined as the predicate whose truth set is the collection of RNA strands where the string **AUC** is a substring (appears inside s , in order and consecutively)

L with domain $S \times \mathbb{Z}^+$ is defined by, for $s \in S$ and $n \in \mathbb{Z}^+$,

$$L(s, n) = \begin{cases} T & \text{if } \text{rnanalen}(s) = n \\ F & \text{otherwise} \end{cases}$$

1. Give a witness for $\exists s_1 \exists s_2 (L(s_1, 4) \wedge L(s_2, 4) \wedge \neg(s_1 = s_2))$ where S is the set of RNA strands, $L(s, n)$ is a predicate with domain $S \times \mathbb{Z}^+$ that is true when s has length n
2. Give a counterexample that disproves $\forall i (\neg L(\text{ACU}, i))$
3. Determine which of the following statements is True or False (you do not need to prove your answer).
 - (a) $\exists n \in \mathbb{Z}^+ \forall s \in S (P_{\text{AUC}}(s) \rightarrow \neg L(s, n))$
 - (b) $\exists n \in \mathbb{Z}^+ \forall s \in S (P_{\text{AUC}}(s) \wedge \neg L(s, n))$
 - (c) $\forall s \in S \exists n \in \mathbb{Z}^+ (P_{\text{AUC}}(s) \vee \neg L(s, n))$
 - (d) $\forall s \in S \exists n \in \mathbb{Z}^+ (L(s, n) \rightarrow \neg P_{\text{AUC}}(s))$

- 9. Predicates and Quantifiers** Express each of the following statements symbolically using quantifiers, variables, propositional connectives, and predicates (make sure to define the domain and the meaning of any predicates you introduce). Then, express its negation as a logically equivalent compound proposition without a $\neg$ in front. Decide whether the statement or its negation is true, and prove it.

- (a) If m is an integer, $m^2 + m + 1$ is odd.
- (b) For all integers n greater than 5, $2^n - 1$ is not prime.
- (c) If n and m are even integers, then $n - m$ is even.

10. Proof strategies Prove each of the following claims. Use only basic definitions and general proof strategies in your proofs; do not use any results proved in class / the textbook as lemmas. You may use basic properties of real numbers that we stated (without proof) in class, e.g. that the difference of two integers is an integer.

- (a) The difference of any rational number and any irrational number is irrational.
- (b) If a, b, c are integers and $a^2 + b^2 = c^2$, then at least one of a and b is even.
- (c) For any integer n , $n^2 + 5$ is not divisible by 4.
- (d) For all positive integers a, b, c , if $a \nmid bc$ then $a \nmid b$. (The notation $a \nmid b$ means “ a does not divide b ”).

Bonus: Watch the video <https://www.youtube.com/watch?v=MhJN9sByRS0> and write down the statement and one or both of the proofs described using the notation, definitions, and proof strategies from CSE 20.

11. Sets Prove or disprove each of the following statements.

- (a) For all sets A and B , $\mathcal{P}(A) \cup \mathcal{P}(B) = \mathcal{P}(A \cup B)$.
- (b) For all sets A and B , $\mathcal{P}(A) \cap \mathcal{P}(B) = \mathcal{P}(A \cap B)$.
- (e) For any sets A, B, C, D , if the Cartesian products $A \times B$ and $C \times D$ are disjoint then either A and C are disjoint or B and D are disjoint (or both).
- (f) There are sets A, B such that $A \in B$ and $A \subseteq B$.
- (g) For all sets A, B, C : $A \cap B = \emptyset$ and $B \cap C = \emptyset$ if and only if $(A \cap B) \cap C = \emptyset$.

12. Induction and Recursion Prove the following statements:

- (a) $\forall s \in S (rnalen(s) = basecount(s, \mathbf{A}) + basecount(s, \mathbf{C}) + basecount(s, \mathbf{U}) + basecount(s, \mathbf{G}))$ where S RNA strands and the functions $rnalen$ and $basecount$ are define recursively as:

$$\begin{array}{ll}
 \text{Basis Step:} & \text{If } b \in B \text{ then} \\
 \text{Recursive Step:} & \text{If } s \in S \text{ and } b \in B, \text{ then}
 \end{array}
 \begin{array}{ll}
 rnalen : S & \rightarrow \mathbb{Z}^+ \\
 rnalen(b) & = 1 \\
 rnalen(sb) & = 1 + rnalen(s)
 \end{array}$$

$$\begin{array}{ll}
 \text{Basis Step:} & \text{If } b_1 \in B, b_2 \in B \\
 \text{Recursive Step:} & \text{If } s \in S, b_1 \in B, b_2 \in B
 \end{array}
 \begin{array}{ll}
 basecount : S \times B & \rightarrow \mathbb{N} \\
 basecount(b_1, b_2) & = \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases} \\
 basecount(sb_1, b_2) & = \begin{cases} 1 + basecount(s, b_2) & \text{when } b_1 = b_2 \\ basecount(s, b_2) & \text{when } b_1 \neq b_2 \end{cases}
 \end{array}$$

- (b) $\forall l_1 \in L \forall l_2 \in L (sum(concat(l_1, l_2)) = sum(l_1) + sum(l_2))$, where the function *concat* is defined as:

$$\begin{array}{lll} & concat : L \times L & \rightarrow L \\ \text{Basis Step:} & \text{If } l \in L & concat([], l) = l \\ \text{Recursive Step:} & \text{If } l, l' \in L \text{ and } n \in \mathbb{N}, \text{ then} & concat((n, l), l') = (n, concat(l, l')) \end{array}$$

and the function $sum : L \rightarrow \mathbb{N}$ that sums all the elements of a list and is defined by:

$$\begin{array}{lll} & sum : L & \rightarrow \mathbb{N} \\ \text{Basis Step:} & sum([]) & = 0 \\ \text{Recursive Step:} & \text{If } l \in L, n \in \mathbb{N} & sum((n, l)) = n + sum(l) \end{array}$$

- 13. Induction and Recursion** At time 0, a particle resides at the point 0 on the real line. Within 1 second, it divides into 2 particles that fly in opposite directions and stop at distance 1 from the original particle. Within the next second, each of these particles again divides into 2 particles flying in opposite directions and stopping at distance 1 from the point of division, and so on. Whenever particles meet they annihilate (leaving nothing behind). How many particles will there be at time $2^{11} - 1$? You do not need to justify your answer. Hint: Derive a formula for the number/ locations of particles at time $2^n - 1$ for arbitrary positive integer n , prove the formula using induction, and apply it when $n = 11$.

- 14. Induction and Recursion** Prove that every positive integer has a base 3 expansion. *Hint: use strong induction.*

- 15. Functions & Cardinalities of sets** Prove each of the following claims.

- (a) For the “function” $f : \mathbb{Z} \rightarrow \{0, 1, 2, 3\}$ given by $f(x) = x \bmod 5$, it is not the case that every element of the domain maps to exactly one element of the codomain (that is, it is not a well-defined function).
- (b) The “function” $f : \mathcal{P}(\mathbb{Z}^+) \rightarrow \mathbb{Z}$ given by

$$f(A) = \text{the maximum element in } A$$

is not well-defined.

- (c) There is a one-to-one function with domain $\{a, b, c\}$ and codomain $\mathbb{R}$.
- (d) There is an onto function with domain R and codomain $\{\pi, \frac{1}{17}\}$, where R is the set of user ratings in a database with 5 movies.
- (e) The Cartesian product $\mathbb{Z}^+ \times \{a, b, c\}$ is countable.
- (f) The interval of real numbers $\{x \in \mathbb{R} \mid 5 \leq x \leq 8\}$ is uncountable. *Hint: you may use the fact that the unit interval $\{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$ is uncountable.*

hw1-definitions-and-notation

CSE20W26

Due: 01/15/26 at 5pm (no penalty late submission until 8am next morning)

In this assignment,

You will practice reading and applying definitions to get comfortable working with mathematical language.

Relevant class material: Week 1.

You will submit this assignment via Gradescope (<https://www.gradescope.com>) in the assignment called “hw1-definitions-and-notation”.

For all HW assignments: These homework assignments may be done individually or in groups of up to four students. Please ensure your name(s) and PID(s) are clearly visible on the first page of your homework submission, start each question on a new page, and upload the PDF to Gradescope. If you’re working in a group, *submit only one submission per group*: one partner uploads the submission through their Gradescope account and then adds the other group member(s) to the Gradescope submission by selecting their name(s) in the “Add Group Members” dialog box. You will need to re-add your group member(s) every time you resubmit a new version of your assignment.

All submitted homework for this class must be typed. You can use a word processing editor if you like (Microsoft Word, Open Office, Notepad, Vim, Google Docs, etc.) but you might find it useful to take this opportunity to learn LaTeX. LaTeX is a markup language used widely in computer science and mathematics. The homework assignments are typed using LaTeX and you can use the source files as templates for typesetting your solutions.

Integrity reminders

- Problems should be solved together, not divided up between the partners. The homework is designed to give you practice with the main concepts and techniques of the course, while getting to know and learn from your classmates.
- Use only resources explicitly allowed for each assignment. Resources not affiliated with this quarter’s version of the class may use inconsistent notation or definitions. When

consulting resources, balance getting the help you need while also protecting the learning experience you will have when the ‘aha’ moments of solving the problem authentically happen.

- Do not share written solutions or partial solutions for homework with other students in the class who are not in your group. Doing so would dilute their learning experience and detract from their success in the class.

When evaluating your homework solutions, we will be considering:

- The correctness of any final answers.
- Whether ideas are presented clearly and logically. You should explain how you arrived at your conclusions, using mathematically sound reasoning. Whether you use formal proof techniques or write a more informal argument for why something is true, your answers should always be well-supported. Your goal should be to convince the reader that your results and methods are sound. A rule of thumb for the level of detail to include is to imagine someone who is taking the class alongside you but missed about a week’s worth of content: does your writeup have enough detail for them to learn from it?

To demonstrate your honest effort in answering homework questions, we expect you to include your attempt to answer *each* part of the question. If you get stuck with your attempt, you can still demonstrate your effort by explaining where you got stuck and what you did to try to get unstuck (e.g. reviewing annotated notes from class, checking the relevant textbook sections, looking up comparable questions on the relevant weekly review quizzes, asking questions on Piazza, attending (instructor/TA/tutor) office hours, consulting generative AI tools, etc.).

Assigned questions

1. Modeling

- (a) In class, we used 4-tuples to represent the user ratings for four movies. This representation is memory-efficient because we use the order of the components in the 4-tuples to represent which movie is being rated. However, it is not easily extended when we want to add new movies to the database. Define a new model that would allow us to represent the user ratings of movie databases where we allow for new movies to be added. Use only the data types we have talked about in class: sets, n -tuples, and strings. Explain the design choices that you used to define your model by referencing properties of the data-type(s) you choose. Demonstrate your model by showing how the rating of the user who dislikes Superman, Wicked and likes KPop Demon Hunters, A Minecraft Movie is represented.

- (b) Colors can be described as amounts of red, green, and blue mixed together ¹ Mathematically, a color can be represented as a 3-tuple (r, g, b) where r represents the red component, g the green component, b the blue component and where each of r, g, b must be a value from this collection of numbers:

{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255}

(This is the same definition as in the Week 1 Review quiz.)

Can you find two different 3-tuples that represent colors that are indistinguishable to your eye? You can use the website in the footnote to play around with different choices of red, green, and blue levels to see if you can distinguish between the resulting colors. Why or why not?

A complete answer will include the specific example 3-tuples that work, along with a description of the colors that they represent and why they are indistinguishable, or an explanation of why there can't be such an example.

2. Sets and functions

- (a) Each of the sets below is described using set builder notation or recursion or as a result of set operations applied to other known sets. Rewrite each of the sets using the roster method.

Remember our discussions of data-types: use clear notation that is consistent with our class notes and definitions to communicate the data-types of the elements in each set.

Sample response that can be used as reference for the detail expected in your answer:

The set $\{A\} \circ \{AU, AC, AG\}$ can be written using the roster method as

$$\{AAU, AAC, AAG\}$$

because set-wise concatenation gives a set whose elements are all possible results of concatenating an element of the left set with an element of the right set. Since the left

¹This RGB representation is common in web applications. Many online tools are available to play around with mixing these colors, e.g. https://www.w3schools.com/colors/colors_rgb.asp.

set in this example only has one element, namely **A**, each of the elements of the set we described starts with **A**. There are three elements of this set, one for each of the distinct elements of the right set.

i.

$$\{n \in \mathbb{Z}^+ \mid n \leq 3\} \times \{m \in \mathbb{N} \mid m \leq 3\}$$

ii. The set X defined recursively as

Basis Step: $1 \in X, 3 \in X, 5 \in X$

Recursive Step: If the integer $n \in X$, then the result of multiplying n by -1 is in X

iii.

$$\{x \in S \mid rnalen(x) = 2\} \circ \{x \in S \mid rnalen(x) = 0\}$$

where S is the set of RNA strands and $rnalen$ is the recursively defined function that we discussed in class,

$$\begin{array}{lll} & & rnalen : S \rightarrow \mathbb{Z}^+ \\ \text{Basis Step:} & \text{If } b \in B \text{ then} & rnalen(b) = 1 \\ \text{Recursive Step:} & \text{If } s \in S \text{ and } b \in B, \text{ then} & rnalen(sb) = 1 + rnalen(s) \end{array}$$

iv.

$$\{(r, g, b) \in C \mid r + g + b = 2 \text{ and } g = 1\}$$

where $C = \{(r, g, b) \mid 0 \leq r \leq 255, 0 \leq g \leq 255, 0 \leq b \leq 255, r \in \mathbb{N}, g \in \mathbb{N}, b \in \mathbb{N}\}$.

(b) Recall the function which takes an ordered pair of ratings 4-tuples and returns a measure of the difference between them

$$d_0 : \{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4 \rightarrow \mathbb{R}$$

given by

$$d_0((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$$

Define a **new** function which we'll call d_{new} with the same domain $\{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4$ and codomain $\mathbb{R}$ but where there is some example pair of ratings 4-tuples

$$((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4))$$

where

$$d_0((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)) \neq d_{new}((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4))$$

Your answer should include **both** a precise and clear definition for the rule defining d_{new} which unambiguously specifies output for each input of the function **and** the example ordered pair of ratings 4-tuples that demonstrate that the functions are not equal. Also include a justification of your answers with (clear, correct, complete) calculations for each of the function applications and/or references to definitions and connecting them with the desired conclusion.

- (c) A function *basecount* that computes the number of a given base b appearing in a RNA strand s is defined recursively:

$$\text{basecount} : S \times B \rightarrow \mathbb{N}$$

Basis Step:

$$\text{If } b_1 \in B, b_2 \in B \quad \text{basecount}((b_1, b_2)) = \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$$

Recursive Step:

$$\text{If } s \in S, b_1 \in B, b_2 \in B \quad \text{basecount}((sb_1, b_2)) = \begin{cases} 1 + \text{basecount}((s, b_2)) & \text{when } b_1 = b_2 \\ \text{basecount}((s, b_2)) & \text{when } b_1 \neq b_2 \end{cases}$$

Consider the function application

$$\text{basecount}((\text{ACAU}, \text{A}))$$

What is the input? What is the output? Give an example of a different choice of input that gives the same output.

Your answer should include clearly labeled answers to each of the three parts of the question, along with a justification for the values of the applications that makes specific reference to the parts of the recursive definition of the *basecount* function used to calculate it.

hw2-numbers Due: 01/29/26 at 5pm (no penalty late submission until 8am next morning)

In this assignment,

You will practice applying functions and tracing algorithms in multiple contexts, and exploring properties of positional number representations.

Relevant class material: Week 2 and Week 3.

You will submit this assignment via Gradescope (<https://www.gradescope.com>) in the assignment called “hw2-numbers”.

For all HW assignments: These homework assignments may be done individually or in groups of up to four students. Please ensure your name(s) and PID(s) are clearly visible on the first page of your homework submission, start each question on a new page, and upload the PDF to Gradescope. If you’re working in a group, *submit only one submission per group*: one partner uploads the submission through their Gradescope account and then adds the other group member(s) to the Gradescope submission by selecting their name(s) in the “Add Group Members” dialog box. You will need to re-add your group member(s) every time you resubmit a new version of your assignment.

All submitted homework for this class must be typed. You can use a word processing editor if you like (Microsoft Word, Open Office, Notepad, Vim, Google Docs, etc.) but you might find it useful to take this opportunity to learn LaTeX. LaTeX is a markup language used widely in computer science and mathematics. The homework assignments are typed using LaTeX and you can use the source files as templates for typesetting your solutions.

Integrity reminders

- Problems should be solved together, not divided up between the partners. The homework is designed to give you practice with the main concepts and techniques of the course, while getting to know and learn from your classmates.
- Use only resources explicitly allowed for each assignment. Resources not affiliated with this quarter’s version of the class may use inconsistent notation or definitions. When consulting resources, balance getting the help you need while also protecting the learning experience you will have when the ‘aha’ moments of solving the problem authentically happen.
- Do not share written solutions or partial solutions for homework with other students in the class who are not in your group. Doing so would dilute their learning experience and detract from their success in the class.

When evaluating your homework solutions, we will be considering:

- The correctness of any final answers.
- Whether ideas are presented clearly and logically. You should explain how you arrived at your conclusions, using mathematically sound reasoning. Whether you use formal proof techniques or write a more informal argument for why something is true, your answers should always be well-supported. Your goal should be to convince the reader that your results and methods are sound. A rule of thumb for the level of detail to include is to imagine someone who is taking the class alongside you but missed about a week's worth of content: does your writeup have enough detail for them to learn from it?

To demonstrate your honest effort in answering homework questions, we expect you to include your attempt to answer *each* part of the question. If you get stuck with your attempt, you can still demonstrate your effort by explaining where you got stuck and what you did to try to get unstuck (e.g. reviewing annotated notes from class, checking the relevant textbook sections, looking up comparable questions on the relevant weekly review quizzes, asking questions on Piazza, attending (instructor/TA/tutor) office hours, consulting generative AI tools, etc.).

Assigned questions

1. Functions and algorithms: in this question you'll explore the properties of the (integer) power and logarithm functions. We'll use some definitions we introduced in class and review quiz, namely the function b^i with domain $\mathbb{Z}^+ \times \mathbb{N}$ and codomain $\mathbb{N}$ defined recursively by

Basis Step:

$$b^0 = 1$$

Recursive Step:

$$\text{If } i \in \mathbb{N}, b^{i+1} = b \cdot b^i$$

and the algorithm:

Calculating integer part of base b logarithm

```

1  procedure logb(b, n: positive integers with  $b > 1$ )
2      i := 0
3      while  $n > b - 1$ 
4          i := i + 1
5          n := n div b
6      return i {i holds the integer part of the base b logarithm of n}

```

- (a) Choose a positive integer b between 3 and 6 (inclusive) and choose a nonnegative integer i between 2 and 5 (inclusive). Demonstrate how to calculate the result of the *logb* algorithm (procedure) when its input is the base b you chose and $n = b^i$ (for the i you choose.) A complete answer will include the specific choice of b and i , along with trace of the calculations of b^i and the computation of the algorithm, including (clear,

correct, complete) calculations for each of the function applications and/or references to definitions and a trace table for algorithm that includes the values of all relevant variables at each iteration (See the annotated Week 2 notes and the review quiz for the level of detail expected in a trace of a function application and a trace table).

- (b) Now we'll go in the other direction: Demonstrate how to calculate the result of 7^y where y is the result of the *logb* algorithm (procedure) when its input is $b = 7$ and $n = 30$. A complete answer will include clearly labelled traces of the calculations of b^i and the computation of the algorithm, including (clear, correct, complete) calculations for each of the function applications and/or references to definitions and a trace table for algorithm that includes the values of all relevant variables at each iteration (See the annotated Week 2 notes for the level of detail expected in a trace of a function application and a trace table).
- (c) Logarithms and powers are supposed to “undo” one another. Explain whether your work in parts (a) and (b) supports that idea, whether you saw anything confusing or surprising about these calculations, and how to explain what you saw.

2. Base expansions

- (a) Pick an integer between 50 and 1000 (inclusive) that you (or one of your group members) came across at some point this week. In a sentence or two, give some context for why you're choosing this number). Write the base expansion of your chosen number in base 2 (binary), base 3 (ternary), base 4, and base 16 (hexadecimal).
- (b) What is the **smallest** width w in which they could write your chosen number in base 8 (octal) fixed-width w ? Justify your answer with reference to the definitions of fixed-width expansions and relevant calculations.
- (c) Express in roster method the set of numbers between 1 and 2 (exclusive) that be written without error (full precision) in binary fixed width expansion with integer part width 3 and fractional part width 2. Justify your answer with reference to the definitions of fixed-width expansions and relevant calculations.
- (d) Consider the strings of 1s that have length 3, 5, 7, and 10. Calculate the numbers

$$[111]_{s,3}$$

$$[11111]_{s,5}$$

$$[1111111]_{s,7}$$

$$[111111111]_{s,10}$$

$$[111]_{2c,3}$$

$$[11111]_{2c,5}$$

$$[1111111]_{2c,7}$$

$$[111111111]_{2c,10}$$

Justify your answers with specific reference to the definitions of sign-magnitude and 2s complement expansions and relevant calculations and discuss any patterns you notice.

3. Multiple representations

Recall that, mathematically, a color can be represented as a 3-tuple (r, g, b) where r represents the red component, g the green component, b the blue component and where each of r, g, b

must be from the collection $\{x \in \mathbb{N} \mid 0 \leq x \leq 255\}$. As an alternative representation, in this assignment we'll use base b fixed-width expansions to represent colors as individual numbers (this definition was introduced in a recent Review quiz question).

Definition: A **hex color** is a nonnegative integer, n , that has a base 16 fixed-width 6 expansion

$$n = (r_1 r_2 g_1 g_2 b_1 b_2)_{16,6}$$

where $(r_1 r_2)_{16,2}$ is the red component, $(g_1 g_2)_{16,2}$ is the green component, and $(b_1 b_2)_{16,2}$ is the blue.

For this question, let's call the set of hex colors HC . In the Week 2 Review quiz, we explored a few different set builder definitions for HC .

- Rewrite the set builder definition of a set below so that it refers to colors rather than numbers: $\{c \in HC \mid c \bmod 256 = 0\}$. A complete answer will justify the new set builder definition by connection with the definition of hex colors and how it impacts the colors that satisfy the specific property for this set.
- Rewrite the set builder definition of a set below so that it refers to colors rather than numbers: $\{c \in HC \mid c < 65536\}$. A complete answer will justify the new set builder definition by connection with the definition of hex colors and how it impacts the colors that satisfy the specific property for this set.
- In art, we can mix two colors to get a new color. For example, red and blue make purple, blue and yellow make green, red and yellow make orange. A mathematical definition of **hex color mixing** would be a function with domain $HC \times HC$ and codomain HC where the result of applying this function to an input (n_1, n_2) is the hex color that results from mixing the hex colors n_1 and n_2 .

Sample work for a related question that can be used as reference for the detail expected in your answer: Consider the attempted mathematical definition $f_1 : HC \times HC \rightarrow HC$ given by

$$f_1((n_1, n_2)) = n_1 + n_2$$

We will show that this function is not well-defined so it cannot give a hex color mixing definition. The reason it is not well-defined is that the application of the rule sometimes gives values that are not in the stated codomain. To see this, we need an example input to f_1 for which the output of the rule is not in HC . Here is one such example: $(n_1, n_2) = ((FFFFFF)_{16,6}, (FFFFFF)_{16,6})$. This example is in $HC \times HC$ because $(FFFFFF)_{16,6}$ is a nonnegative integer that has a base 16 fixed-width 6 expansion so it is a hex color. We now apply the definition of the rule in f_1 :

$$\begin{aligned} f_1((n_1, n_2)) &= f_1((FFFFFF)_{16,6}, (FFFFFF)_{16,6}) = (FFFFFF)_{16,6} + (FFFFFF)_{16,6} \\ &= 2(15 \cdot 16^5 + 15 \cdot 16^4 + 15 \cdot 16^3 + 15 \cdot 16^2 + 15 \cdot 16^1 + 15 \cdot 16^0) \\ &= 2 \cdot 15 \cdot (1048576 + 65536 + 4096 + 256 + 16 + 1) = 2 \cdot 15 \cdot 1118481 = 33554430 \end{aligned}$$

This is not a hex color because it is greater than or equal to $16^6 = 16777216$, so (using the Week 2 notes on which numbers can be represented with fixed-width expansions) it does not have a hexadecimal **fixed width 6** expansion.

In this question, we'll look at another attempted mathematical definition for hex color mixing. We define $f_2 : HC \times HC \rightarrow HC$ given by

$$f_2((n_1, n_2)) = (n_1 + n_2) \text{ div } 2$$

This is a well-defined function. You do not need to prove this or hand it in, but it's good practice to make sure you understand why it's well-defined. Notice that the function application

$$f_2(((FF0000)_{16,6}, (FFFE00)_{16,6}))$$

can be calculated as:

$$\begin{aligned} f_2(((FF0000)_{16,6}, (FFFE00)_{16,6})) &= ((FF0000)_{16,6} + (FFFE00)_{16,6}) \text{ div } 2 \\ &= ((15 \cdot 16^5 + 15 \cdot 16^4 + 0 \cdot 16^3 + 0 \cdot 16^2 + 0 \cdot 16^1 + 0 \cdot 16^0) \\ &\quad + ((15 \cdot 16^5 + 15 \cdot 16^4 + 15 \cdot 16^3 + 14 \cdot 16^2 + 0 \cdot 16^1 + 0 \cdot 16^0))) \text{ div } 2 \\ &= (16711680 + 16776704) \text{ div } 2 = 33488384 \text{ div } 2 = 16744192 \end{aligned}$$

Since this is less than $16^6 = 16777216$, we can represent it using base 16 fixed-width 6:.

$$(16744192)_{10} = 15 \cdot 16^5 + 15 \cdot 16^4 + 7 \cdot 16^3 + 15 \cdot 16^2 + 0 \cdot 16^1 + 0 \cdot 16^0 = (FF7F00)_{16,6}$$

Using the web tool https://www.w3schools.com/colors/colors_rgb.asp, we can verify that $(FF0000)_{16,6}$ is red, $(FFFE00)_{16,6}$ is yellow, and $(FF7F00)_{16,6}$ is orange.

Show that f_2 does not work as a hex color mixing definition by finding another ordered pair of hex colors for which the result of applying f_2 does not give the expected hex color. Include the numerical description of each colors you mention, alongside a description of them in English. Describe how you know what these colors are (if you use a web color tool, include its URL in your submission writeup; if not, describe your reasoning). Justify your example with (clear, correct, complete) calculations and/or references to definitions, and connecting them with the desired conclusion.

4. Fixed-width addition.

- (a) Choose example width 5 first summand and width 5 second summand so that in the binary fixed-width addition (adding one bit at time, using the usual column-by-column and carry arithmetic, and ignoring the carry from the leftmost column), the example satisfies all three conditions below simultaneously
 - (1) When interpreting each of the summands and the result in binary fixed-width 5, the result represents the actual value of the sum of the summands **and**
 - (2) when interpreting each of the summands and the sum in sign-magnitude width 5, the result represents the actual value of the sum of the summands **and**

- (3) when interpreting each of the summands and the sum in 2s complement width 5, the result represents the actual value of the sum of the summands.
- (b) Choose an example width 5 first summand and second summand so that in the binary fixed-width addition (adding one bit at time, using the usual column-by-column and carry arithmetic, and ignoring the carry from the leftmost column), the example satisfies all three conditions below simultaneously
- (1) When interpreting each of the summands and the result in binary fixed-width 5, the result **does not** represent the actual value of the sum of the summands **and**
 - (2) when interpreting each of the summands and the sum in sign-magnitude width 5, the result **does not** represents the actual value of the sum of the summands **and**
 - (3) when interpreting each of the summands and the sum in 2s complement width 5, the result represents the actual value of the sum of the summands.

A complete solution will clearly specify each summand and the result of binary fixed-width addition with this choice of summands; will specify the value of each summand and the result for binary fixed-width 5, sign-magnitude width 5, and 2s complement width 5 (and include calculations connecting with the definitions of these representations to explain these values); and a conclusion connecting the calculations to the properties laid out in the question.

hw3-circuits-and-logic Due: 02/12/26 at 5pm (late submission until 8am next morning)

In this assignment, you will consider how circuits and logic can be used to represent mathematical claims. You will use propositional operators to express and evaluate these claims. You will analyze statements and determine if they are true or false using valid proof strategies.

Relevant class material: Week 3, 4, and 5.

You will submit this assignment via Gradescope (<https://www.gradescope.com>) in the assignment called “hw3-circuits-and-logic”.

For all HW assignments: These homework assignments may be done individually or in groups of up to four students. Please ensure your name(s) and PID(s) are clearly visible on the first page of your homework submission, start each question on a new page, and upload the PDF to Gradescope. If you’re working in a group, *submit only one submission per group*: one partner uploads the submission through their Gradescope account and then adds the other group member(s) to the Gradescope submission by selecting their name(s) in the “Add Group Members” dialog box. You will need to re-add your group member(s) every time you resubmit a new version of your assignment.

All submitted homework for this class must be typed. You can use a word processing editor if you like (Microsoft Word, Open Office, Notepad, Vim, Google Docs, etc.) but you might find it useful to take this opportunity to learn LaTeX. LaTeX is a markup language used widely in computer science and mathematics. The homework assignments are typed using LaTeX and you can use the source files as templates for typesetting your solutions.

Integrity reminders

- Problems should be solved together, not divided up between the partners. The homework is designed to give you practice with the main concepts and techniques of the course, while getting to know and learn from your classmates.
- Use only resources explicitly allowed for each assignment. Resources not affiliated with this quarter’s version of the class may use inconsistent notation or definitions. When consulting resources, balance getting the help you need while also protecting the learning experience you will have when the ‘aha’ moments of solving the problem authentically happen.
- Do not share written solutions or partial solutions for homework with other students in the class who are not in your group. Doing so would dilute their learning experience and detract from their success in the class.

When evaluating your homework solutions, we will be considering:

- The correctness of any final answers.

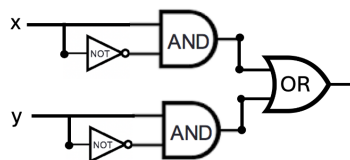
- Whether ideas are presented clearly and logically. You should explain how you arrived at your conclusions, using mathematically sound reasoning. Whether you use formal proof techniques or write a more informal argument for why something is true, your answers should always be well-supported. Your goal should be to convince the reader that your results and methods are sound. A rule of thumb for the level of detail to include is to imagine someone who is taking the class alongside you but missed about a week's worth of content: does your writeup have enough detail for them to learn from it?

To demonstrate your honest effort in answering homework questions, we expect you to include your attempt to answer *each* part of the question. If you get stuck with your attempt, you can still demonstrate your effort by explaining where you got stuck and what you did to try to get unstuck (e.g. reviewing annotated notes from class, checking the relevant textbook sections, looking up comparable questions on the relevant weekly review quizzes, asking questions on Piazza, attending (instructor/TA/tutor) office hours, consulting generative AI tools, etc.).

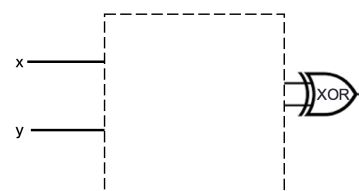
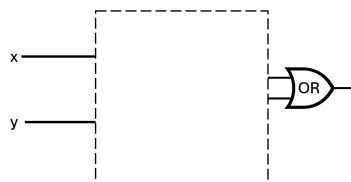
Assigned questions

1. Circuits.

- (a) Consider the circuit below with inputs x and y . Can one of the AND gates be exchanged with the OR gate without changing the input-output table of the circuit? If so, write out the input-output table that results, and briefly explain why this exchange works. If not, explain why not with reference to the definitions of the logic gates.



- (b) Is there a way to fill in the blank portion of the two logic circuits below *with the same gates connected in the same way* so that the resulting circuits have the same input-output value *even though* one uses an OR gate at the end and the other uses an XOR gate? If so, design the circuit that would be used, write out the input-output table that results, and briefly explain why your design works. If not, explain why not with reference to the definitions of the logic gates.



2. Compound propositions. The set of strings of length 4 whose characters are 0s or 1s is the result of four successive set-wise concatenations: $\{0, 1\} \circ \{0, 1\} \circ \{0, 1\} \circ \{0, 1\}$. Let's call this set X_4 . Consider the function $f : X_4 \rightarrow X_4$ defined by

$$f(x) = \begin{cases} y & \text{when } (x)_{2,4} < 15 \text{ and } (y)_{2,4} = (x)_{2,4} + 1 \\ 1111 & \text{when } x = 1111 \end{cases}$$

for each $x \in X_4$. In other words, we can describe the function as: f takes a string, interprets it as the binary fixed-width 4 expansion of an integer, and then adds 1 to that integer (unless x is already representing the greatest integer that can be represented in binary fixed-width 4) and outputs the binary fixed-width 4 expansion of the result.

- (a) Fill in the blanks in the following input-output definition table with four inputs x_3, x_2, x_1, x_0 and four outputs y_3, y_2, y_1, y_0 so that $f(x_3x_2x_1x_0) = y_3y_2y_1y_0$. Remember to explain all your answers.

x_3	x_2	x_1	x_0	y_3	y_2	y_1	y_0
1	1	1	1	1	1	BLANK1	1
1	1	1	0	1	1	1	1
1	1	0	1	1	BLANK2	1	0
1	1	0	0	1	1	0	1
1	0	1	1	1	1	0	0
1	0	1	0	1	0	1	1
1	0	0	1	1	0	1	0
1	0	0	0	BLANK3	0	0	1
0	1	1	1	1	0	0	0
0	1	1	0	0	1	1	BLANK4
0	1	0	1	0	1	1	BLANK5
0	1	0	0	0	1	0	1
0	0	1	1	0	1	0	0
0	0	1	0	0	0	1	1
0	0	0	1	0	0	1	0
0	0	0	0	BLANK6	0	0	1

- (b) Construct an expression (as a compound proposition) for each of y_0, y_1, y_2 , and y_3 in terms of the inputs x_3, x_2, x_1, x_0 . Justify your expression by referring to the definition of the logic gates XOR, AND, OR, NOT and the definition of the function f . Hint: our work on the half-adder might be helpful.
- (c) Draw a combinatorial circuit corresponding to two (or more) of these (output) compound propositions. Remember that the symbols for the inputs will be on the left-hand-side and the symbol for the outputs y_0 and/or y_1 and/or y_2 and/or y_3 will be on the right-hand side. Use gates (draw the appropriate shapes and add labels for clarity) and wires to connect the inputs appropriately to give the output.

3. Logical Equivalence. Imagine a friend suggests the following argument to you: “The compound proposition

$$(x \vee y) \wedge z$$

is logically equivalent to

$$x \vee (y \wedge z)$$

because I can transform one to the other using the following sequence of logical equivalences:

$$(x \vee y) \wedge z \equiv (x \vee (y \wedge y)) \wedge z \equiv x \vee ((y \vee y) \wedge z) \equiv x \vee (y \wedge z)$$

because y is logically equivalent to both $y \wedge y$ and to $y \vee y$ ”.

- (a) Prove to your friend that they made a mistake by giving a truth assignment to the propositional variables x, y, z so that the two compound propositions $(x \vee y) \wedge z$ and $x \vee (y \wedge z)$ have different truth values. Justify your choice by evaluating these compound propositions using the definitions of the logical connectives and include enough intermediate steps so that a student in CSE 20 who may be struggling with the material can still follow along with your reasoning.
- (b) Help your friend find the problem in their argument by pointing out which step(s) were incorrect and why.
- (c) Give **three** different compound propositions that are actually logically equivalent to (and not the same as)

$$(x \vee y) \wedge z$$

Justify each one of these logical equivalences either by applying a sequence of logical equivalences or using a truth table. Notice that you can use other logical operators (e.g. $\neg, \vee, \wedge, \oplus, \rightarrow, \leftrightarrow$) when constructing your compound propositions.

Bonus; not for credit (do not hand in): How would you translate each of the equivalent compound propositions in English? Does doing so help illustrate why they are equivalent?

4. Evaluating predicates. Consider the following predicates, each of which has as its domain the set of all bitstrings whose leftmost bit is 1

$E(x)$ is T exactly when $(x)_2$ is even, and is F otherwise

$L(x)$ is T exactly when $(x)_2 < 15$, and is F otherwise

$M(x)$ is T exactly when $(x)_2 > 2$ and is F otherwise.

Sample response that can be used as reference for the detail expected in your answer: To prove that the statement

$$\forall x L(x)$$

is false, we can use the counterexample $x = 1111$, which is a bitstring whose leftmost bit is 1 (so is in the domain). Applying the definition of $L(x)$, since $(1111)_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 2 + 1 = 15$ which is not (strictly) less than 15, we have that $L(1111) = F$ and so the universal statement is false.

-
- (a) Use a counterexample to prove that the statement

$$\forall x(L(x) \rightarrow E(x))$$

is false. Your proof needs to include references to the relevant definitions.

- (b) Use a witness to prove that the statement

$$\exists x(L(x) \wedge M(x))$$

is true. Your proof needs to include references to the relevant definitions.

- (c) Translate each of the statements in the previous two parts to English.

5. Set properties. Let $W = \mathcal{P}(\{1, 2, 3, 4, 5\})$.

Sample response that can be used as reference for the detail expected in your answer for parts (a) and (b) below:

To give an example element in the set $\{X \in W \mid 1 \in X\} \cap \{X \in W \mid 2 \in X\}$, consider $\{1, 2\}$. To prove that this is in the set, by definition of intersection, we need to show that $\{1, 2\} \in \{X \in W \mid 1 \in X\}$ and that $\{1, 2\} \in \{X \in W \mid 2 \in X\}$.

- By set builder notation, elements in $\{X \in W \mid 1 \in X\}$ have to be elements of W which have 1 as an element. By definition of power set, elements of W are subsets of $\{1, 2, 3, 4, 5\}$. Since each element in $\{1, 2\}$ is an element of $\{1, 2, 3, 4, 5\}$, $\{1, 2\}$ is a subset of $\{1, 2, 3, 4, 5\}$ and hence is an element of W . Also, by roster method, $1 \in \{1, 2\}$. Thus, $\{1, 2\}$ satisfies the conditions for membership in $\{X \in W \mid 1 \in X\}$.
 - Similarly, by set builder notation, elements in $\{X \in W \mid 2 \in X\}$ have to be elements of W which have 2 as an element. By definition of power set, elements of W are subsets of $\{1, 2, 3, 4, 5\}$. Since each element in $\{1, 2\}$ is an element of $\{1, 2, 3, 4, 5\}$, $\{1, 2\}$ is a subset of $\{1, 2, 3, 4, 5\}$ and hence is an element of W . Also, by roster method, $2 \in \{1, 2\}$. Thus, $\{1, 2\}$ satisfies the conditions for membership in $\{X \in W \mid 2 \in X\}$.
-

(a) Give two (different) example elements in

$$W \times \mathcal{P}(W)$$

Justify your examples by explanations that include references to the relevant definitions.

(b) Give one example element in

$$\{X \in W \mid (1 \in X) \wedge (X \cap \{3, 4\} = \emptyset)\}$$

Justify your example by explanations that include references to the relevant definitions.

(c) Consider the following statement and attempted proof:

$$\forall A \in W \forall B \in W (((A \cap B) \subseteq A) \rightarrow (A \subseteq B))$$

- (1) Towards a universal generalization argument, **choose arbitrary** $A \in W, B \in W$.
- (2) We need **to show** $((A \cap B) \subseteq A) \rightarrow (A \subseteq B)$.
- (3) Towards a proof of the conditional by the contrapositive, **assume** $A \subseteq B$, and we need **to show** that $(A \cap B) \subseteq A$.
- (4) By definition of subset inclusion, this means we need **to show** $\forall x (x \in A \cap B \rightarrow x \in A)$.
- (5) Towards a universal generalization, **choose arbitrary** x ; we need **to show** that $x \in A \cap B \rightarrow x \in A$.

- (6) Towards a direct proof, **assume** $x \in A \cap B$, and we need **to show** $x \in A$.
 - (7) By definition of set intersection and set builder notation, we have that $x \in A \wedge x \in B$.
 - (8) By the definition of conjunction, $x \in A$, which is what we needed to show.
- QED

Demonstrate that this attempted proof is invalid by providing and justifying a **counterexample** (disproving the statement). Then, explain why this attempted proof is invalid by identifying in which step a definition or proof strategy is used incorrectly, and describing how the definition or proof strategy was misused.

In your proofs and disproofs of statements above, justify each step by reference to a component of the following proof strategies we have discussed so far, and/or to relevant definitions and calculations.

- A counterexample can be used to prove that $\forall x P(x)$ is **false**.
- A witness can be used to prove that $\exists x P(x)$ is **true**.
- **Proof of universal by exhaustion:** To prove that $\forall x P(x)$ is true when P has a finite domain, evaluate the predicate at **each** domain element to confirm that it is always T.
- **Proof by universal generalization:** To prove that $\forall x P(x)$ is true, we can take an arbitrary element e from the domain and show that $P(e)$ is true, without making any assumptions about e other than that it comes from the domain.
- To prove that $\exists x P(x)$ is **false**, write the universal statement that is logically equivalent to its negation and then prove it true using universal generalization.
- **Strategies for conjunction:** To prove that $p \wedge q$ is true, have two subgoals: subgoal (1) prove p is true; and, subgoal (2) prove q is true. To prove that $p \wedge q$ is false, it's enough to prove that p is false. To prove that $p \wedge q$ is false, it's enough to prove that q is false.
- **Proof of Conditional by Direct Proof:** To prove that the implication $p \rightarrow q$ is true, we can assume p is true and use that assumption to show q is true.
- **Proof of Conditional by Contrapositive Proof:** To prove that the implication $p \rightarrow q$ is true, we can assume $\neg q$ is true and use that assumption to show $\neg p$ is true.
- **Proof by Cases:** To prove q when we know $p_1 \vee p_2$, show that $p_1 \rightarrow q$ and $p_2 \rightarrow q$.

hw4-proofs-sets-recursion Due: 02/26/2026 at 5pm (late submission until 8am next morning)

In this assignment, you will analyze statements and determine if they are true or false using valid proof strategies. You will also determine if candidate arguments are valid. You will work with recursively defined sets and functions and prove properties about them, practicing induction and other proof strategies.

Relevant class material: Weeks 5, 6, and 7.

You will submit this assignment via Gradescope (<https://www.gradescope.com>) in the assignment called “hw4-proofs-sets-recursion”.

For all HW assignments: These homework assignments may be done individually or in groups of up to four students. Please ensure your name(s) and PID(s) are clearly visible on the first page of your homework submission, start each question on a new page, and upload the PDF to Gradescope. If you’re working in a group, *submit only one submission per group*: one partner uploads the submission through their Gradescope account and then adds the other group member(s) to the Gradescope submission by selecting their name(s) in the “Add Group Members” dialog box. You will need to re-add your group member(s) every time you resubmit a new version of your assignment.

All submitted homework for this class must be typed. You can use a word processing editor if you like (Microsoft Word, Open Office, Notepad, Vim, Google Docs, etc.) but you might find it useful to take this opportunity to learn LaTeX. LaTeX is a markup language used widely in computer science and mathematics. The homework assignments are typed using LaTeX and you can use the source files as templates for typesetting your solutions.

Integrity reminders

- Problems should be solved together, not divided up between the partners. The homework is designed to give you practice with the main concepts and techniques of the course, while getting to know and learn from your classmates.
- Use only resources explicitly allowed for each assignment. Resources not affiliated with this quarter’s version of the class may use inconsistent notation or definitions. When consulting resources, balance getting the help you need while also protecting the learning experience you will have when the ‘aha’ moments of solving the problem authentically happen.
- Do not share written solutions or partial solutions for homework with other students in the class who are not in your group. Doing so would dilute their learning experience and detract from their success in the class.

When evaluating your homework solutions, we will be considering:

- The correctness of any final answers.
- Whether ideas are presented clearly and logically. You should explain how you arrived at your conclusions, using mathematically sound reasoning. Whether you use formal proof techniques or write a more informal argument for why something is true, your answers should always be well-supported. Your goal should be to convince the reader that your results and methods are sound. A rule of thumb for the level of detail to include is to imagine someone who is taking the class alongside you but missed about a week's worth of content: does your writeup have enough detail for them to learn from it?

To demonstrate your honest effort in answering homework questions, we expect you to include your attempt to answer **each** part of the question. If you get stuck with your attempt, you can still demonstrate your effort by explaining where you got stuck and what you did to try to get unstuck (e.g. reviewing annotated notes from class, checking the relevant textbook sections, looking up comparable questions on the relevant weekly review quizzes, asking questions on Piazza, attending (instructor/TA/tutor) office hours, consulting generative AI tools, etc.).

In your proofs and disproofs of statements below, justify each step by reference to a component of the following proof strategies we have discussed so far, and/or to relevant definitions and calculations.

- A counterexample can be used to prove that $\forall x P(x)$ is **false**.
- A witness can be used to prove that $\exists x P(x)$ is **true**.
- **Proof of universal by exhaustion:** To prove that $\forall x P(x)$ is true when P has a finite domain, evaluate the predicate at **each** domain element to confirm that it is always T.
- **Proof by universal generalization:** To prove that $\forall x P(x)$ is true, we can take an arbitrary element e from the domain and show that $P(e)$ is true, without making any assumptions about e other than that it comes from the domain.
- To prove that $\exists x P(x)$ is **false**, write the universal statement that is logically equivalent to its negation and then prove it true using universal generalization.

- **Strategies for conjunction:** To prove that $p \wedge q$ is true, have two subgoals: subgoal (1) prove p is true; and, subgoal (2) prove q is true. To prove that $p \wedge q$ is false, it's enough to prove that p is false. To prove that $p \wedge q$ is false, it's enough to prove that q is false.
- **Proof of Conditional by Direct Proof:** To prove that the implication $p \rightarrow q$ is true, we can assume p is true and use that assumption to show q is true.
- **Proof of Conditional by Contrapositive Proof:** To prove that the implication $p \rightarrow q$ is true, we can assume $\neg q$ is true and use that assumption to show $\neg p$ is true.
- **Proof by Cases:** To prove q when we know $p_1 \vee p_2$, show that $p_1 \rightarrow q$ and $p_2 \rightarrow q$.
- **Proof by Structural Induction:** To prove that $\forall x \in X P(x)$ where X is a recursively defined set, prove two cases:
 - Basis Step: Show the statement holds for elements specified in the basis step of the definition.
 - Recursive Step: Show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.
- **Proof by Mathematical Induction:** To prove a universal quantification over the set of all integers greater than or equal to some base integer b :
 - Basis Step: Show the statement holds for b .
 - Recursive Step: Consider an arbitrary integer n greater than or equal to b , assume (as the **induction hypothesis**) that the property holds for n , and use this and other facts to prove that the property holds for $n + 1$.
- **Proof by Strong Induction** To prove that a universal quantification over the set of all integers greater than or equal to some base integer b holds, pick a fixed nonnegative integer j and then:
 - Basis Step: Show the statement holds for $b, b + 1, \dots, b + j$.
 - Recursive Step: Consider an arbitrary integer n greater than or equal to $b + j$, assume (as the **strong induction hypothesis**) that the property holds for **each of** $b, b + 1, \dots, n$, and use this and other facts to prove that the property holds for $n + 1$.
- **Proof by Contradiction**

To prove that a statement p is true, pick another statement r and once we show that $\neg p \rightarrow (r \wedge \neg r)$ then we can conclude that p is true.

Informally The statement we care about can't possibly be false, so it must be true.

Assigned questions

1. Number properties. Consider the predicate $F(a, b) = \text{"}a \text{ is a factor of } b\text{"}$ over the domain $\mathbb{Z}^{\neq 0} \times \mathbb{Z}$. Consider the following quantified statements

- | | |
|--|---|
| (i) $\forall x \in \mathbb{Z} (F(1, x))$ | (v) $\forall x \in \mathbb{Z}^{\neq 0} \exists y \in \mathbb{Z} (F(x, y))$ |
| (ii) $\forall x \in \mathbb{Z}^{\neq 0} (F(x, 1))$ | (vi) $\exists x \in \mathbb{Z}^{\neq 0} \forall y \in \mathbb{Z} (F(x, y))$ |
| (iii) $\exists x \in \mathbb{Z} (F(1, x))$ | (vii) $\forall y \in \mathbb{Z} \exists x \in \mathbb{Z}^{\neq 0} (F(x, y))$ |
| (iv) $\exists x \in \mathbb{Z}^{\neq 0} (F(x, 1))$ | (viii) $\exists y \in \mathbb{Z} \forall x \in \mathbb{Z}^{\neq 0} (F(x, y))$ |

- (a) Which of the statements (i) - (viii) is being **proved** by the following proof:

By universal generalization, **choose** e to be an **arbitrary** integer. We need to show that $F(1, e)$. By definition of the predicate F , we can rewrite this goal as $\exists c \in \mathbb{Z} (e = c \cdot 1)$. We pick the **witness** $c = e$, which is an integer and therefore in the domain. Calculating, $c \cdot 1 = e \cdot 1 = e$, as required. Since the predicate $F(1, e)$ evaluates to true for the arbitrary integer e , the claim has been proved. $\square$

Hint: it may be useful to identify the key words in the proof that indicate proof strategies.

- (b) Which of the statements (i) - (viii) is being **disproved** by the following proof:

To disprove the statement, we need to find a counterexample. We choose 2, a nonzero integer so in the domain. We need to show that $\neg F(2, 1)$. By definition of the predicate F , we can rewrite this goal as $1 \bmod 2 \neq 0$. By definition of integer division, since $1 = 0 \cdot 2 + 1$ (and $0 \leq 1 < 2$), $1 \bmod 2 = 1$ which is nonzero so the counterexample works to disprove the original statement. $\square$

Hint: it may be useful to identify the key words in the proof that indicate proof strategies.

- (c) Translate the statement to English, state whether is it true or false, and then justify your answer (by proving the statement or its negation). (*Graded for correctness of translation and evaluation of statement (is it true or false?) and fair effort completeness of the proof*)

$$\exists x \in \mathbb{Z}^{\neq 0} \exists y \in \mathbb{Z}^{\neq 0} (\neg(x = y) \wedge F(x, y) \wedge F(y, x))$$

- (d) Translate the statement to English, state whether is it true or false, and then justify your answer (by proving the statement or its negation). (*Graded for correctness of translation and evaluation of statement (is it true or false?) and fair effort completeness of the proof*)

$$\forall x \in \mathbb{Z}^{\neq 0} \forall y \in \mathbb{Z}^{\neq 0} (F(x, y) \rightarrow \neg F(y, x))$$

- (e) Translate the statement to English, state whether is it true or false, and then justify your answer (by proving the statement or its negation) (*Graded for correctness of translation*

and evaluation of statement (is it true or false?) and fair effort completeness of the proof)

$$\exists x \in \mathbb{Z}^{\neq 0} \exists y \in \mathbb{Z} (F((x, y)) \wedge F((x + 1, y)) \wedge F((x + 2, y)))$$

- (f) Translate the statement to English, state whether is it true or false, and then justify your answer (by proving the statement or its negation). (*Graded for correctness of translation and evaluation of statement (is it true or false?) and fair effort completeness of the proof*)

$$\forall x \in \mathbb{Z}^{\neq 0} (F((x, x^2)) \wedge F((x, x^3)))$$

2. Structural induction. Recall that we define the set of bases as $B = \{\mathbf{A}, \mathbf{C}, \mathbf{U}, \mathbf{G}\}$. The set of RNA strands S is defined (recursively) by:

$$\begin{array}{ll} \text{Basis Step:} & \mathbf{A} \in S, \mathbf{C} \in S, \mathbf{U} \in S, \mathbf{G} \in S \\ \text{Recursive Step:} & \text{If } s \in S \text{ and } b \in B, \text{ then } sb \in S \end{array}$$

where sb is string concatenation. The function $rnalen$ that computes the length of RNA strands in S is defined recursively by $rnalen : S \rightarrow \mathbb{Z}^+$

Basis step: If $b \in B$ then $rnalen(b) = 1$

Recursive step: If $s \in S$ and $b \in B$, then $rnalen(sb) = 1 + rnalen(s)$

The function $basecount$ that computes the number of a given base b appearing in a RNA strand s is defined recursively:

$$\begin{array}{ll} & basecount : S \times B \rightarrow \mathbb{N} \\ \text{Basis Step:} & \text{If } b_1 \in B, b_2 \in B \quad basecount((b_1, b_2)) = \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases} \\ \text{Recursive Step:} & \text{If } s \in S, b_1 \in B, b_2 \in B \quad basecount((sb_1, b_2)) = \begin{cases} 1 + basecount((s, b_2)) & \text{when } b_1 = b_2 \\ basecount((s, b_2)) & \text{when } b_1 \neq b_2 \end{cases} \end{array}$$

- (a) Translate the statement to English, state whether is it true or false, and then justify your answer (by proving the statement or its negation). (*Graded for correctness of translation and evaluation of statement (is it true or false?) and fair effort completeness of the proof*)

$$\forall b \in B \exists s \in S (rnalen(s) = basecount((s, b)))$$

- (b) Translate the statement to English, state whether is it true or false, and then justify your answer (by proving the statement or its negation). (*Graded for correctness of translation and evaluation of statement (is it true or false?) and fair effort completeness of the proof*)

$$\forall s \in S (rnalen(s) > basecount((s, \mathbf{A})))$$

- (c) Translate the statement to English, state whether is it true or false, and then justify your answer (by proving the statement or its negation). (*Graded for correctness of translation and evaluation of statement (is it true or false?) and fair effort completeness of the proof*)

$$\forall s \in S \forall b \in B \text{ (basecount}((s, b)) > 0)$$

3. Games and induction. The game of Nim-Var is a two-player game (which is a variant of Nim). At the start of the game, there are two piles, each containing n jelly beans (n is a positive integer). On a player's turn, that player picks one of the two piles and does **one** of the following: either

- removes some positive number of jelly beans from that pile, **or**
- moves some positive number of jelly beans (that is less than the current total number of jelly beans in the pile) from that pile to the other. *Note: if there is only one jelly bean left in a pile, the player cannot move this jelly bean to the other pile.*

The player to take the last jelly bean wins. Use strong induction to prove that the second player always has a winning strategy in Nim-Move. A complete and correct solution will first identify what the strategy is, and then prove that following this strategy will lead the second player to win the game (no matter what the first player chooses to do at each turn).

4. Linked Lists. Recall the recursive definition of the set of linked lists of natural numbers (from class)

Basis Step: $[] \in L$

Recursive Step: If $l \in L$ and $n \in \mathbb{N}$, then $(n, l) \in L$

and the definitions of the function which gives the length of a linked list of natural numbers $length : L \rightarrow \mathbb{N}$

Basis Step: $length([]) = 0$

Recursive Step: If $l \in L$ and $n \in \mathbb{N}$, then $length((n, l)) = 1 + length(l)$

the function $append : L \times \mathbb{N} \rightarrow L$ that adds an element at the end of a linked list

Basis Step: If $m \in \mathbb{N}$ then $append([], m) = (m, [])$

Recursive Step: If $l \in L$ and $n \in \mathbb{N}$ and $m \in \mathbb{N}$, then $append(((n, l), m)) = (n, append((l, m)))$

and the function $prepend : L \times \mathbb{N} \rightarrow L$ that adds an element at the front of a linked list

$$prepend((l, n)) = (n, l)$$

Sample response that can be used as reference for the detail expected in your answer: To evaluate the result of $length((4, (2, (7, []))))$ we calculate:

$$\begin{aligned}
 length((4, (2, (7, [])))) &= 1 + length((2, (7, []))) \\
 &\quad \text{using recursive step of definition of } length, \text{ with } n = 4, l = (2, (7, [])) \\
 &= 1 + 1 + length((7, [])) \\
 &\quad \text{using recursive step of definition of } length, \text{ with } n = 2, l = (7, []) \\
 &= 1 + 1 + 1 + length([]) \\
 &\quad \text{using recursive step of definition of } length, \text{ with } n = 7, l = [] \\
 &= 1 + 1 + 1 + 0 \\
 &\quad \text{using basis step of definition of } length, \text{ since input to } length \text{ is } []
 \end{aligned}$$

In this question, we'll consider the combination of these functions with a new function, one that removes the element at the front of the list (if there is any). We define $remove : L \rightarrow L$ by

Basis Step: $remove([]) = []$
Recursive Step: If $l \in L$ and $n \in \mathbb{N}$, then $remove((n, l)) = l$

- (a) What is the result of $remove(append(prepend(((10, []), 5), 20)))$? For full credit, include all intermediate steps with brief justifications for each.
- (b) Prove the statement

$$\forall l \in L (remove(prepend((l, 0))) = l)$$

- (c) Disprove the statement

$$\forall l \in L (remove(append((l, 0))) = l)$$

5. Primes, divisors, and proof strategies. For each statement below, identify the **main logical structure or connective** of the statement, list the proof strategies that could be used to prove and to disprove a statement with that structure, then identify whether the statement is true or false and justify with a proof of the statement or its negation. (*Graded for correctness of identification of logical structure and proof strategies and evaluation of statement (is it true or false?) and fair effort completeness of the proof*)

Sample response that can be used as reference for the detail expected in your answer:

Consider the statement: There is a greatest negative integer.

The main logical structure for this statement is that it is an **existential** statement, as we can see by translating it to symbols:

$$\exists g \in \mathbb{Z}^- \forall x \in \mathbb{Z}^- (g \geq x)$$

To prove an existential statement, the main proof strategy we could use is to find a witness. Proof by cases and proof by contradiction could also be used, because both can be used to prove any statement (no matter its logical structure).

To disprove an existential statement, we would need to prove its negation, which (using De-Morgan's Laws) can be written as a universal statement. Therefore, to disprove this statement the strategies we could use are universal generalization or structural induction (because $\mathbb{Z}^-$ is a recursively defined set) or proof by cases or proof by contradiction. Notice that proof by exhaustion is not possible because the domain is not finite.

The statement is true, as we can see from the witness $g = -1$, since it is in the domain $\mathbb{Z}^-$ and when we evaluate

$$\forall x \in \mathbb{Z}^- (-1 \geq x)$$

we can proceed by universal generalization and take an arbitrary negative integer x , which by definition means $x < 0$, and since x is an integer, guarantees $x \leq -1 = g$, as required.

- (a) The quotient of any even number with any nonzero even number is even.
- (b) There are two odd numbers (not necessarily distinct) whose sum is even.
- (c) The greatest common divisor of 5 and 23 is 1 and the greatest common divisor of 7 and 19 is 1.
- (d) There are two positive integers greater than 20 that have the same greatest common divisor with 20.

hw5-cardinality-functions-relations Due: 03/12/2026 at 5pm (late submission until 8am next morning)

In this assignment, you will work with cardinality, functions, and binary relations and prove properties about them, practicing proof strategies and applications.

Relevant class material: Weeks 8, 9, 10.

You will submit this assignment via Gradescope (<https://www.gradescope.com>) in the assignment called “hw5-cardinality-functions-relations”.

For all HW assignments: These homework assignments may be done individually or in groups of up to four students. Please ensure your name(s) and PID(s) are clearly visible on the first page of your homework submission, start each question on a new page, and upload the PDF to Gradescope. If you’re working in a group, *submit only one submission per group*: one partner uploads the submission through their Gradescope account and then adds the other group member(s) to the Gradescope submission by selecting their name(s) in the “Add Group Members” dialog box. You will need to re-add your group member(s) every time you resubmit a new version of your assignment.

All submitted homework for this class must be typed. You can use a word processing editor if you like (Microsoft Word, Open Office, Notepad, Vim, Google Docs, etc.) but you might find it useful to take this opportunity to learn LaTeX. LaTeX is a markup language used widely in computer science and mathematics. The homework assignments are typed using LaTeX and you can use the source files as templates for typesetting your solutions.

Integrity reminders

- Problems should be solved together, not divided up between the partners. The homework is designed to give you practice with the main concepts and techniques of the course, while getting to know and learn from your classmates.
- Use only resources explicitly allowed for each assignment. Resources not affiliated with this quarter’s version of the class may use inconsistent notation or definitions. When consulting resources, balance getting the help you need while also protecting the learning experience you will have when the ‘aha’ moments of solving the problem authentically happen.
- Do not share written solutions or partial solutions for homework with other students in the class who are not in your group. Doing so would dilute their learning experience and detract from their success in the class.

When evaluating your homework solutions, we will be considering:

- The correctness of any final answers.

- Whether ideas are presented clearly and logically. You should explain how you arrived at your conclusions, using mathematically sound reasoning. Whether you use formal proof techniques or write a more informal argument for why something is true, your answers should always be well-supported. Your goal should be to convince the reader that your results and methods are sound. A rule of thumb for the level of detail to include is to imagine someone who is taking the class alongside you but missed about a week's worth of content: does your writeup have enough detail for them to learn from it?

To demonstrate your honest effort in answering homework questions, we expect you to include your attempt to answer **each** part of the question. If you get stuck with your attempt, you can still demonstrate your effort by explaining where you got stuck and what you did to try to get unstuck (e.g. reviewing annotated notes from class, checking the relevant textbook sections, looking up comparable questions on the relevant weekly review quizzes, asking questions on Piazza, attending (instructor/TA/tutor) office hours, consulting generative AI tools, etc.).

In your proofs and disproofs of statements below, justify each step by reference to a component of the following proof strategies we have discussed so far, and/or to relevant definitions and calculations.

- A counterexample can be used to prove that $\forall x P(x)$ is **false**.
- A witness can be used to prove that $\exists x P(x)$ is **true**.
- **Proof of universal by exhaustion:** To prove that $\forall x P(x)$ is true when P has a finite domain, evaluate the predicate at **each** domain element to confirm that it is always T.
- **Proof by universal generalization:** To prove that $\forall x P(x)$ is true, we can take an arbitrary element e from the domain and show that $P(e)$ is true, without making any assumptions about e other than that it comes from the domain.
- To prove that $\exists x P(x)$ is **false**, write the universal statement that is logically equivalent to its negation and then prove it true using universal generalization.

- **Strategies for conjunction:** To prove that $p \wedge q$ is true, have two subgoals: subgoal (1) prove p is true; and, subgoal (2) prove q is true. To prove that $p \wedge q$ is false, it's enough to prove that p is false. To prove that $p \wedge q$ is false, it's enough to prove that q is false.
- **Proof of Conditional by Direct Proof:** To prove that the implication $p \rightarrow q$ is true, we can assume p is true and use that assumption to show q is true.
- **Proof of Conditional by Contrapositive Proof:** To prove that the implication $p \rightarrow q$ is true, we can assume $\neg q$ is true and use that assumption to show $\neg p$ is true.
- **Proof by Cases:** To prove q when we know $p_1 \vee p_2$, show that $p_1 \rightarrow q$ and $p_2 \rightarrow q$.
- **Proof by Structural Induction:** To prove that $\forall x \in X P(x)$ where X is a recursively defined set, prove two cases:
 - Basis Step: Show the statement holds for elements specified in the basis step of the definition.
 - Recursive Step: Show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.
- **Proof by Mathematical Induction:** To prove a universal quantification over the set of all integers greater than or equal to some base integer b :
 - Basis Step: Show the statement holds for b .
 - Recursive Step: Consider an arbitrary integer n greater than or equal to b , assume (as the **induction hypothesis**) that the property holds for n , and use this and other facts to prove that the property holds for $n + 1$.
- **Proof by Strong Induction** To prove that a universal quantification over the set of all integers greater than or equal to some base integer b holds, pick a fixed nonnegative integer j and then:
 - Basis Step: Show the statement holds for $b, b + 1, \dots, b + j$.
 - Recursive Step: Consider an arbitrary integer n greater than or equal to $b + j$, assume (as the **strong induction hypothesis**) that the property holds for **each of** $b, b + 1, \dots, n$, and use this and other facts to prove that the property holds for $n + 1$.
- **Proof by Contradiction**

To prove that a statement p is true, pick another statement r and once we show that $\neg p \rightarrow (r \wedge \neg r)$ then we can conclude that p is true.

Informally The statement we care about can't possibly be false, so it must be true.

Assigned questions

1. Binary relations. In the review quiz, we considered the binary relation on $\mathbb{Z}^+$ defined by

$$\{(a, b) \mid \exists c \in \mathbb{Z}(b = ac)\}$$

Let's call that relation R_1 . Consider the following other binary relations on $\mathbb{Z}^+$:

$$R_2 = \{(a, b) \in \mathbb{Z}^+ \times \mathbb{Z}^+ \mid \gcd(a, b) = 1\}$$

$$R_3 = \{(a, b) \in \mathbb{Z}^+ \times \mathbb{Z}^+ \mid a + b \leq 2026\}$$

Sample response that can be used as reference for the detail expected in your answer: To prove that R_1 is not symmetric we need to find a counterexample to

$$\forall x \in \mathbb{Z}^+ \forall y \in \mathbb{Z}^+ ((x, y) \in R_1 \rightarrow (y, x) \in R_1)$$

Consider $x = 2$ and $y = 4$. Then $(x, y) \in R_1$ because $\exists c \in \mathbb{Z}(y = xc)$ is witnessed by the integer $c = 2$, since $4 = 2 \cdot 2$. However, $\exists c \in \mathbb{Z}(x = yc)$ is not true: to prove this, we consider an arbitrary integer c and consider the two (exhaustive) cases $c \geq 1$ and $c < 1$. In case 1, assume $c \geq 1$, and then $yc = 4c \geq 4 > 3$ so $yc \neq 2 = x$. In case 2, assume $c < 1$ so $c \leq 0$, and then $yc = 4c \leq 0$ so is not equal to 2, which is x . Thus, we've found positive integers x and y where $(x, y) \in R_1$ and $(y, x) \notin R_1$ so R_1 is not symmetric.

- (a) Give example positive integers that are related according to each of these three relations. In other words, give example values (a_1, b_1) , (a_2, b_2) , and (a_3, b_3) such that $(a_i, b_i) \in R_i$ for each $i = 1, 2, 3$.
 - (b) Prove that R_2 is not transitive.
 - (c) Prove that R_3 is not reflexive.
 - (d) Prove that R_2 is symmetric.
 - (e) Prove that R_3 is not anti-symmetric.
 - (f) Give an example relation on $\mathbb{Z}^+$ that is both symmetric and anti-symmetric. Briefly justify your example.
2. Equivalence relations. Recall that we say a is **congruent to $b \bmod n$** means $(a, b) \in R_{(\bmod n)}$, that is $(a \bmod n = b \bmod n)$. A common notation is to write this as $a \equiv b(\bmod n)$.

A **modular inverse** of an integer x relative to modulus n (where n is a positive number and x is an integer between 0 and $n - 1$, inclusive) is defined to be an integer y between 0 and $n - 1$ (inclusive) such that $xy \equiv 1(\bmod n)$.

- (a) Fill in the two multiplication tables below. In the left table, each cell in row labelled a and column labelled b is an integer p in the set $\{0, 1, 2, 3, 4\}$ satisfying $ab \equiv p \pmod{5}$. In the right table, each cell in row labelled a and column labelled b is an integer p in the set $\{0, 1, 2, 3, 4, 5\}$ satisfying $ab \equiv p \pmod{6}$.

	0	1	2	3	4
0	0	0	0	0	0
1	0				
2	0				
3	0				
4	0				

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					
5	0					

- (b) Does each integer x between 1 and 4 (inclusive) have a modular inverse relative to modulus 5? Does each integer y between 1 and 5 (inclusive) have a modular inverse relative to modulus 6? Justify your answers using appropriate proof strategies and definitions.
- (c) Is there an example of integers x_1, x_2 between 1 and 4 (inclusive) so that $x_1 x_2 \equiv 0 \pmod{5}$? Is there an example of integers y_1, y_2 between 1 and 5 (inclusive) so that $y_1 y_2 \equiv 0 \pmod{6}$? Justify your answers using appropriate proof strategies and definitions.
3. Partial orders. Recall the recursive definition of the set of linked lists of natural numbers (from class)

Basis Step: $[] \in L$

Recursive Step: If $l \in L$ and $n \in \mathbb{N}$, then $(n, l) \in L$

and the definition of the function which gives the length of a linked list of natural numbers $length : L \rightarrow \mathbb{N}$

Basis Step: $length([]) = 0$

Recursive Step: If $l \in L$ and $n \in \mathbb{N}$, then $length((n, l)) = 1 + length(l)$

For this question, we'll restrict our attention to just linked lists with data in the set $\{0, 1\}$ which have length 0, 1, or 2. Let's call this restricted set of lists L' .

Note: L' has 7 distinct elements.

Define two different partial orders with domain L' . For each of the partial orders, you can specify its definition by listing the set of ordered pairs with roster method, giving a set builder definition, giving a recursive definition, or using a clear and precise English description of which lists are related to one another. Then, draw the Hasse diagram for each of the two partial orders you defined.

4. Equivalence classes and partitions. Recall that in a movie recommendation system, each user's ratings of movies is represented as a n -tuple (with the positive integer n being the number of movies in the database), and each component of the n -tuple is an element of the collection $\{-1, 0, 1\}$. Assume there are five movies in the database, so that each user's ratings can be

represented as a 5-tuple. We call Rt_5 the set of all ratings 5-tuples. Consider the binary relation on the set of all 5-tuples where each component of the 5-tuple is an element of the collection $\{-1, 0, 1\}$:

$$G = \{(u, v) \in Rt_5 \times Rt_5 \mid \text{the number of 1s in } u \text{ is the same as the number of 1s in } v\}$$

This is an equivalence relation (you do not need to prove this).

Recall that the **equivalence class** of an element $x \in X$ for an equivalence relation $\sim$ on the set X is the set $\{s \in X \mid (x, s) \in \sim\}$. We write this as $[x]_\sim$.

- (a) Find a ratings 5-tuple v such that $[v]_G = \{v\}$. Justify your choice of v .
 - (b) Find distinct ratings 5-tuples u_1, u_2 ($u_1 \neq u_2$) whose equivalence classes $[u_1]_G$ and $[u_2]_G$ have the same size.
 - (c) Find distinct ratings 5-tuples w_1, w_2 ($w_1 \neq w_2$) whose equivalence classes $[w_1]_G$ and $[w_2]_G$ have different sizes.
5. Modular exponentiation. Imagine you are playing the role of Alice in the Diffie Hellman key agreement (exchange) protocol. You and Bob have agreed to use the prime $p = 7$ and its primitive root $a = 3$. Your secret integer is $k_1 = 3$.

- (a) Calculate the number you send to Bob, $a^{k_1} \bmod p$. Use the modular exponentiation algorithm for the calculation. Include a trace of the algorithm in your solution.

Modular Exponentiation

```

1  procedure modular_exponentiation(b: integer;
2                                     n = (ak-1ak-2...a1a0)2, m: positive integers)
3  x := 1
4  power := b mod m
5  for i := 0 to k - 1
6      if ai = 1 then x := (x · power) mod m
7      power := (power · power) mod m
8  return x {x equals bn mod m}
```

- (b) Bob sends you the number 5. Compute your shared key, $(a^{k_2})^{k_1} \bmod p$. Hint: $a^{k_2} \bmod p$ is what Bob sent you. Include all relevant calculations, annotated with explanations, for full credit.
- (c) What are some possible values for Bob's secret integer? What algorithm are you using to compute them?